Trees

Binary Tree.

→ At most 2 child nodes.

(Binary Tree).

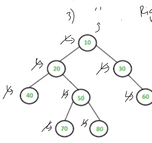
```
Node {
    int data;
    Node* left;
    Node* right;
}
```

(General Tree).

```
Node {
    int data;
    vector<node*> children;
}
```

Preorder

→ 1) Visit the current node.
2) Preorder left subtree
3) " Right "

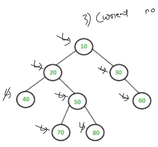


TC: O(n)
AS: O(H).

[10, 20, 40, 50, 70, 80, 30, 60]

Postorder

1) Left subtree.
2) Right subtree.
3) Current node.



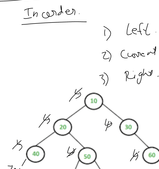
TC: O(n).
AS: O(H).

[40, 70, 80, 50, 20, 60, 30, 10]

[7, 8, 3, 6, 5, 2, 1]

Incorder

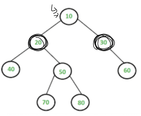
1) Left.
2) Current.
3) Right.



TC: O(n)
AS: O(H)

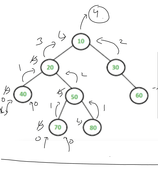
[40, 20, 70, 50, 80, 10, 30, 60].

Height.



left subtree Ht = 3
right subtree Ht = 2

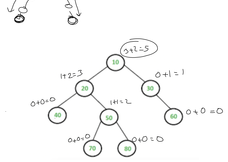
Ht = 1 + max (left Ht, right Ht)



```
int heightNode(node root) {
    if (root == NULL) {
        return 0;
    }
    int leftHeight = height(root->left);
    int rightHeight = height(root->right);
    return 1 + max(leftHeight, rightHeight);
}
```

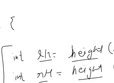
Diameter.

diam = 10.
lH = 5
rH = 5



```
diam {
    int lH = height (root → left).
    int rH = height (root → right).
    result = max (diam, lH + rH)
}
diam (root → left()
diam (root → right).
```

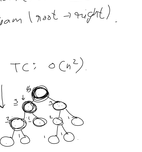
TC: O(n²)



6

⇒ TC: O(n)
AS: O(H) + O(n)
= O(n).

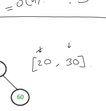
Level-order Trav.



[10, 20, 30, 40, 50, 60, 70, 80]

40, 50, 60.

2. 70, 70, 80, 70, 50, 60, 70, 80

[10, 20, 30, 10, 50, 60, 70, 80].

TC: O(n).
AS: O(n).